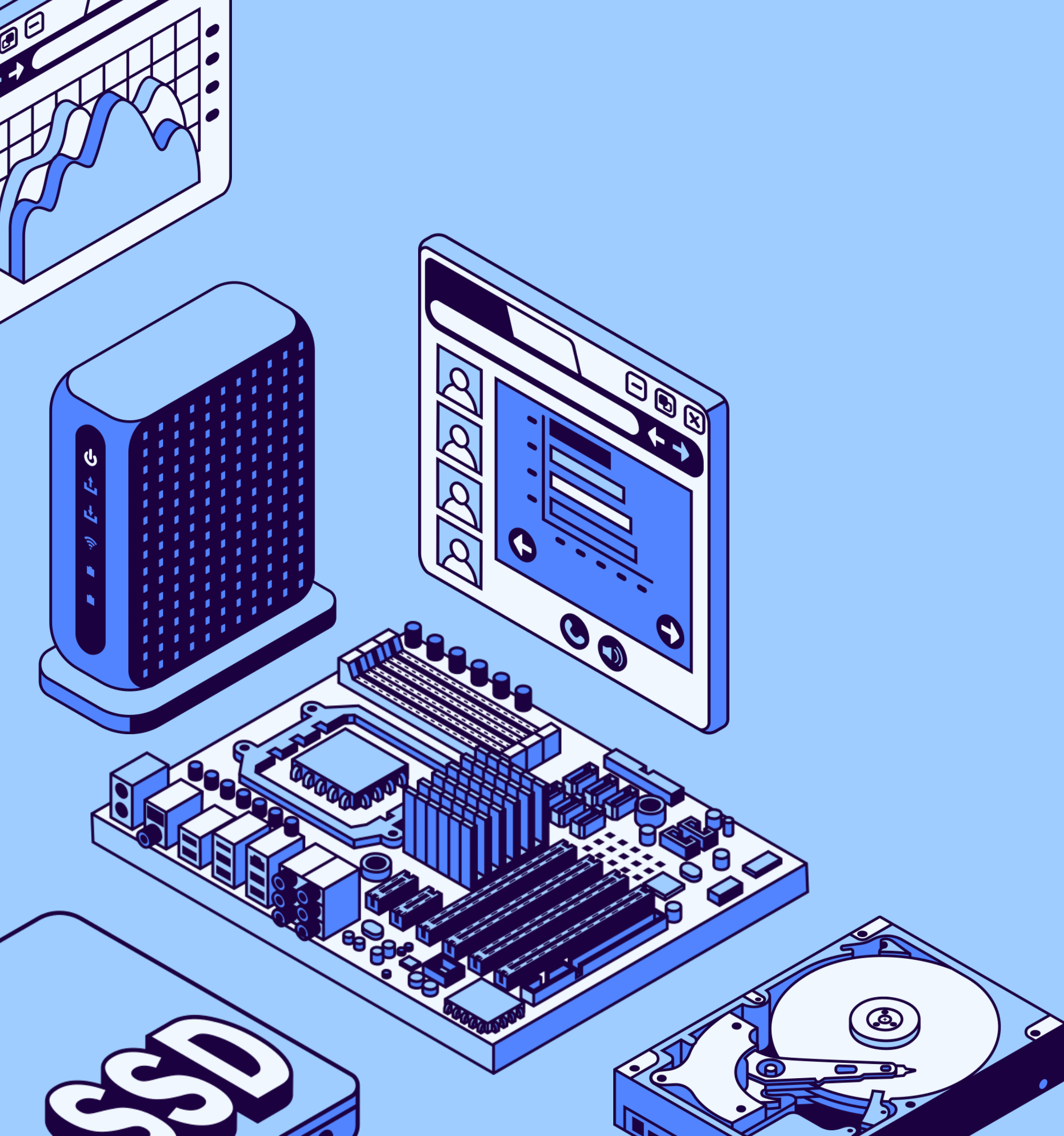
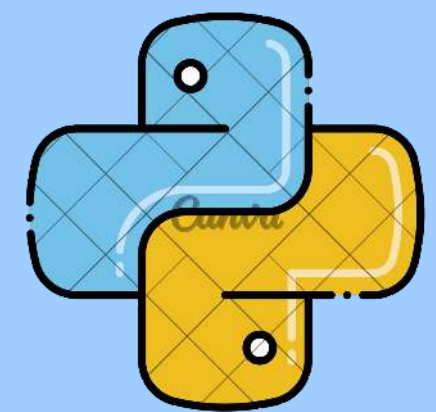
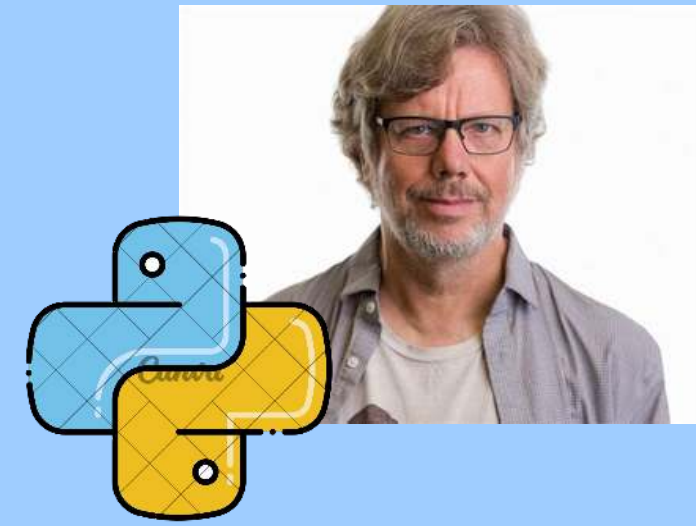


Bases de Python

POR JAIME CIRIACO



Python



Lenguaje de programación muy popular, creado en 1991 por Guido van Rossum. Se caracteriza por ser:

Sencillo y legible: su sintaxis es clara y se parece bastante al lenguaje natural.

Multiparadigma: soporta programación orientada a objetos, programación funcional y estructurada.

Versátil: se usa en muchos campos: desarrollo web, análisis de datos, inteligencia artificial, automatización, videojuegos, ciencia, etc.

Multiplataforma: funciona en Windows, macOS y Linux.

Con gran comunidad y librerías: existen miles de módulos y frameworks que permiten ampliar sus capacidades.

¿Por qué Python?

Gran comunidad y cantidad
de librerías

Principal lenguaje para ia y big
data

Arregla tareas con menos
texto

Muy bueno con los números:
soporte, análisis, n° infinitos, n°
complejos, ...

**When you switch
from C++ to Python**



¿Qué versión?

Python 2 ya es legacy

Trabajaremos con Python 3

Buscar Python.org



Yo y los k-bros saliendo de nuestra primera clase de programación

"Presentar un triangulo rectangulo en pantalla con for"

"suma de dos numeros"

"Hacer el mayor de dos numeros"

$K=K+1$

$i++$



Después de un mes estudiando programación

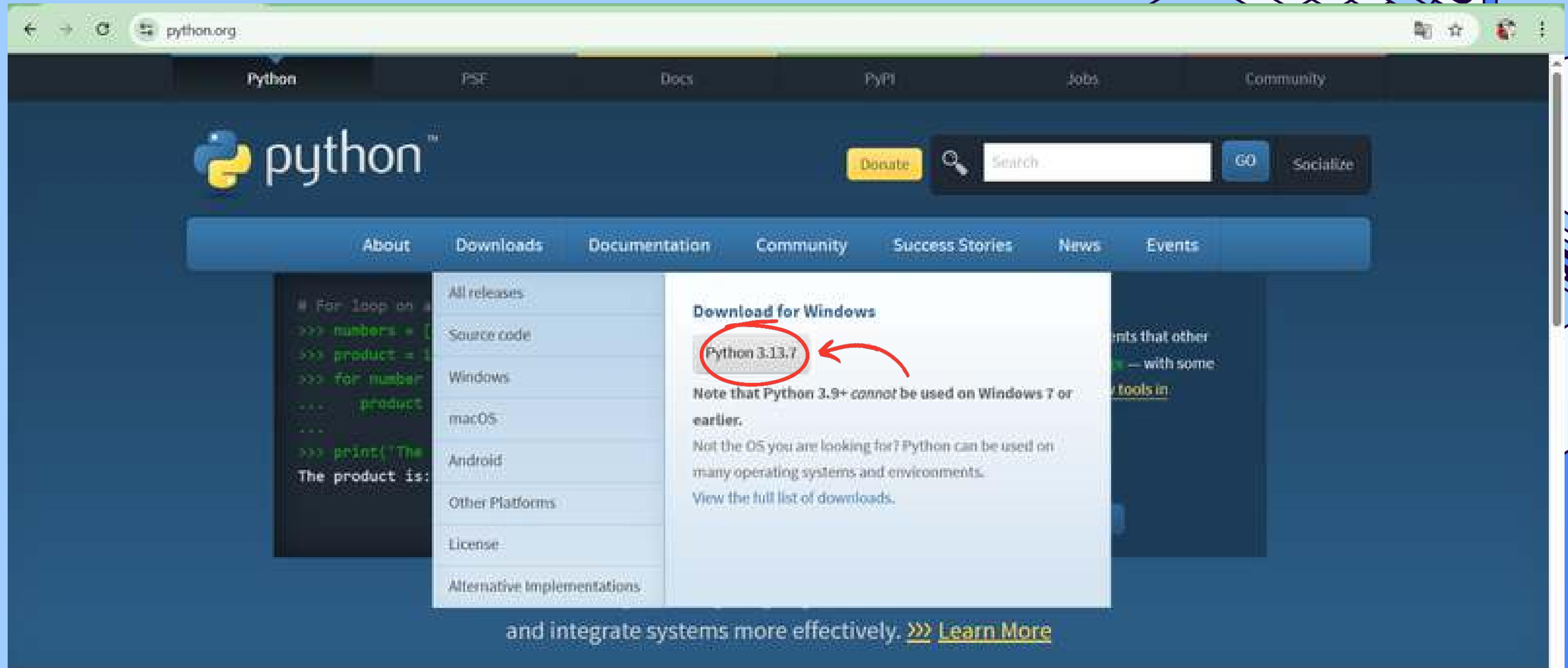
"Arboles y nodos" "Punteros y manejo directo de la memoria"

"POO"

"Estructuras de datos"



PASO 1



The screenshot shows the Python.org website with the 'Downloads' menu open. The 'Python 3.13.7' link is circled in red, and a red arrow points to it. The 'Download for Windows' section is also visible, providing instructions for Windows users.

python.org

Python PyPI Docs Jobs Community

python™

Donate Search GO Socialize

About Downloads Documentation Community Success Stories News Events

All releases
Source code
Windows
macOS
Android
Other Platforms
License
Alternative Implementations

Download for Windows

Python 3.13.7

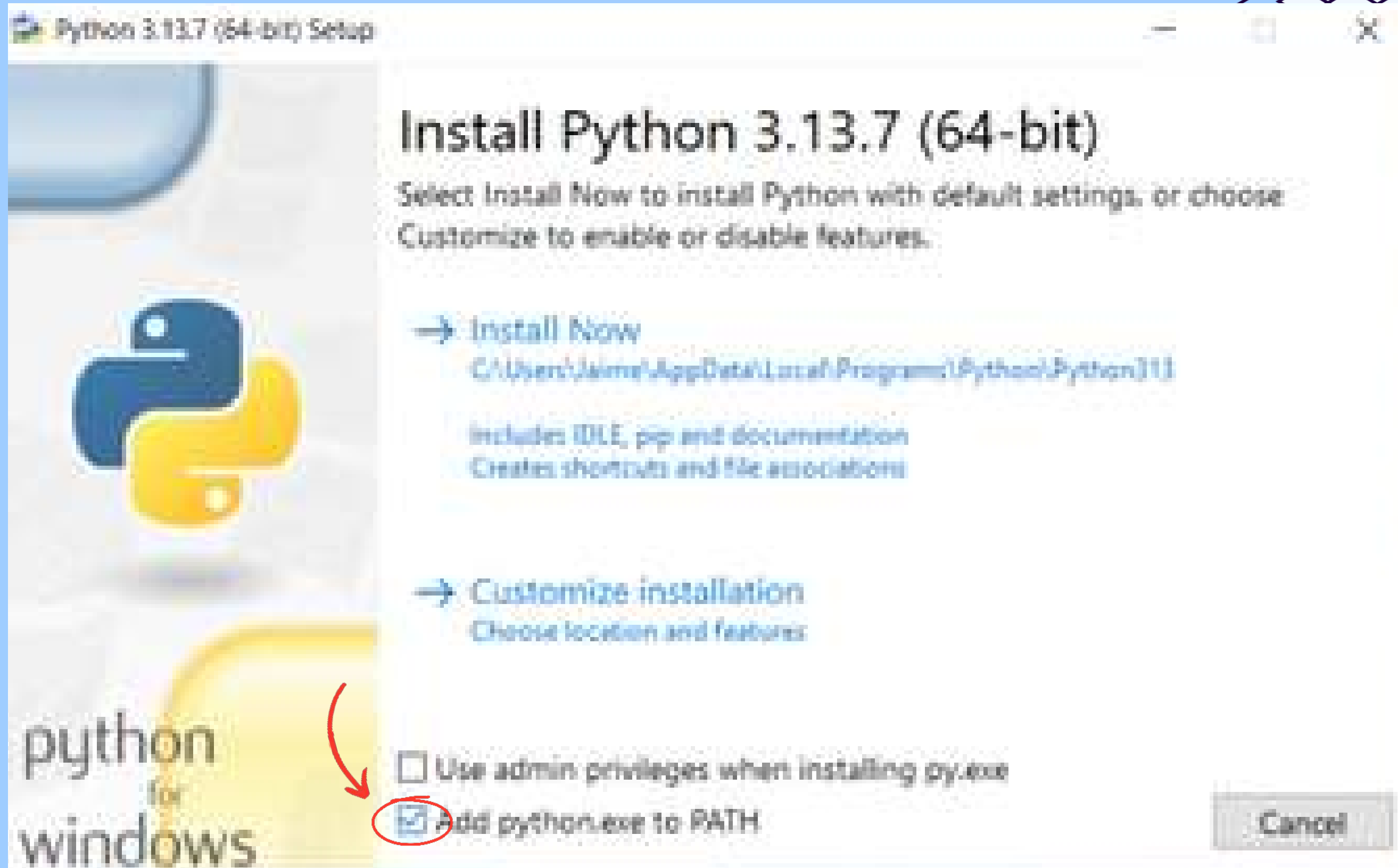
Note that Python 3.9+ cannot be used on Windows 7 or earlier.

Not the OS you are looking for? Python can be used on many operating systems and environments.

[View the full list of downloads.](#)

and integrate systems more effectively. [»» Learn More](#)

PASO 2



PASO 3



```
Símbolo del sistema
Microsoft Windows [Versión 10.0.19045.6332]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Jaime>python --version
Python 3.13.7

C:\Users\Jaime>
```

PASO 4

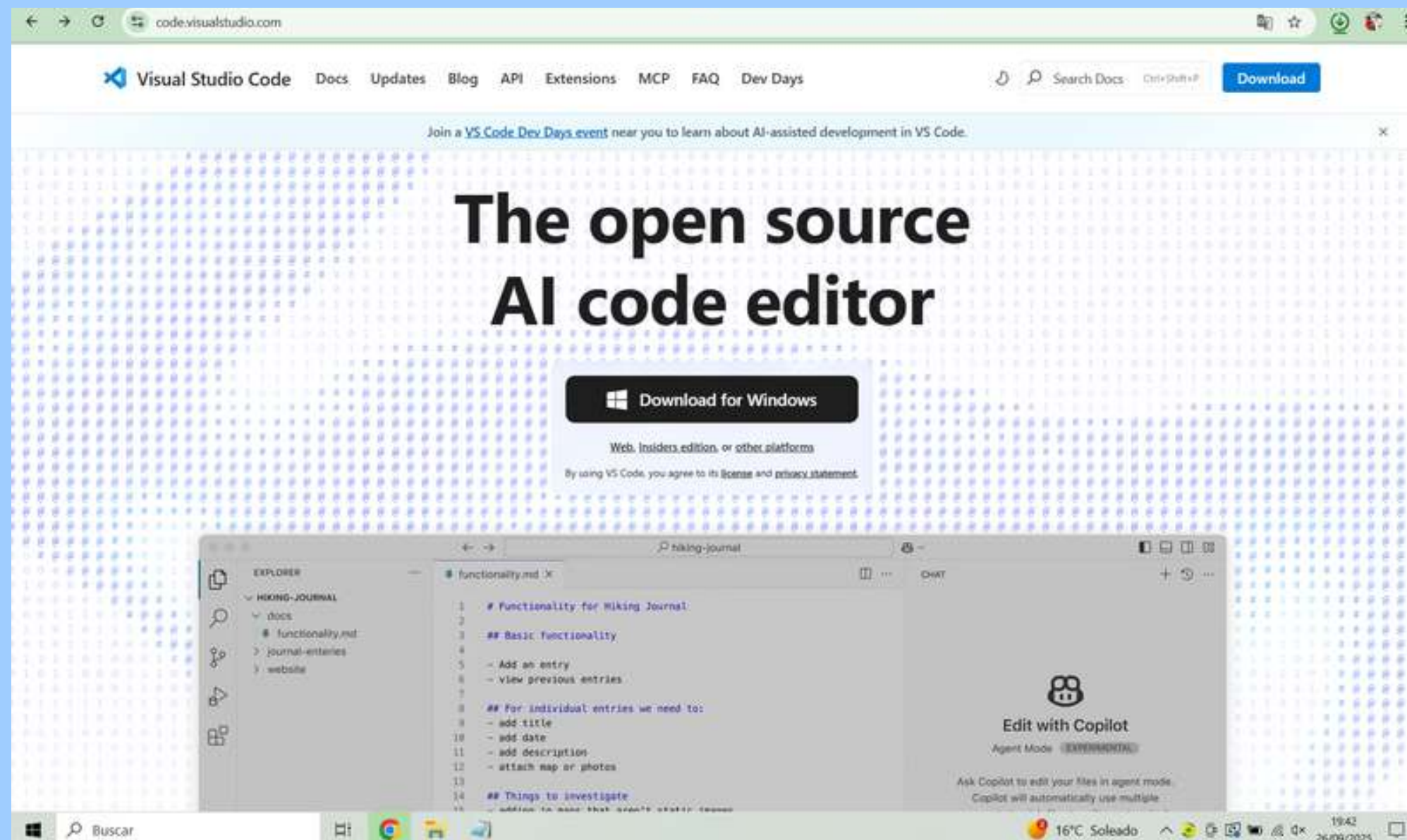
editor de video o IDE



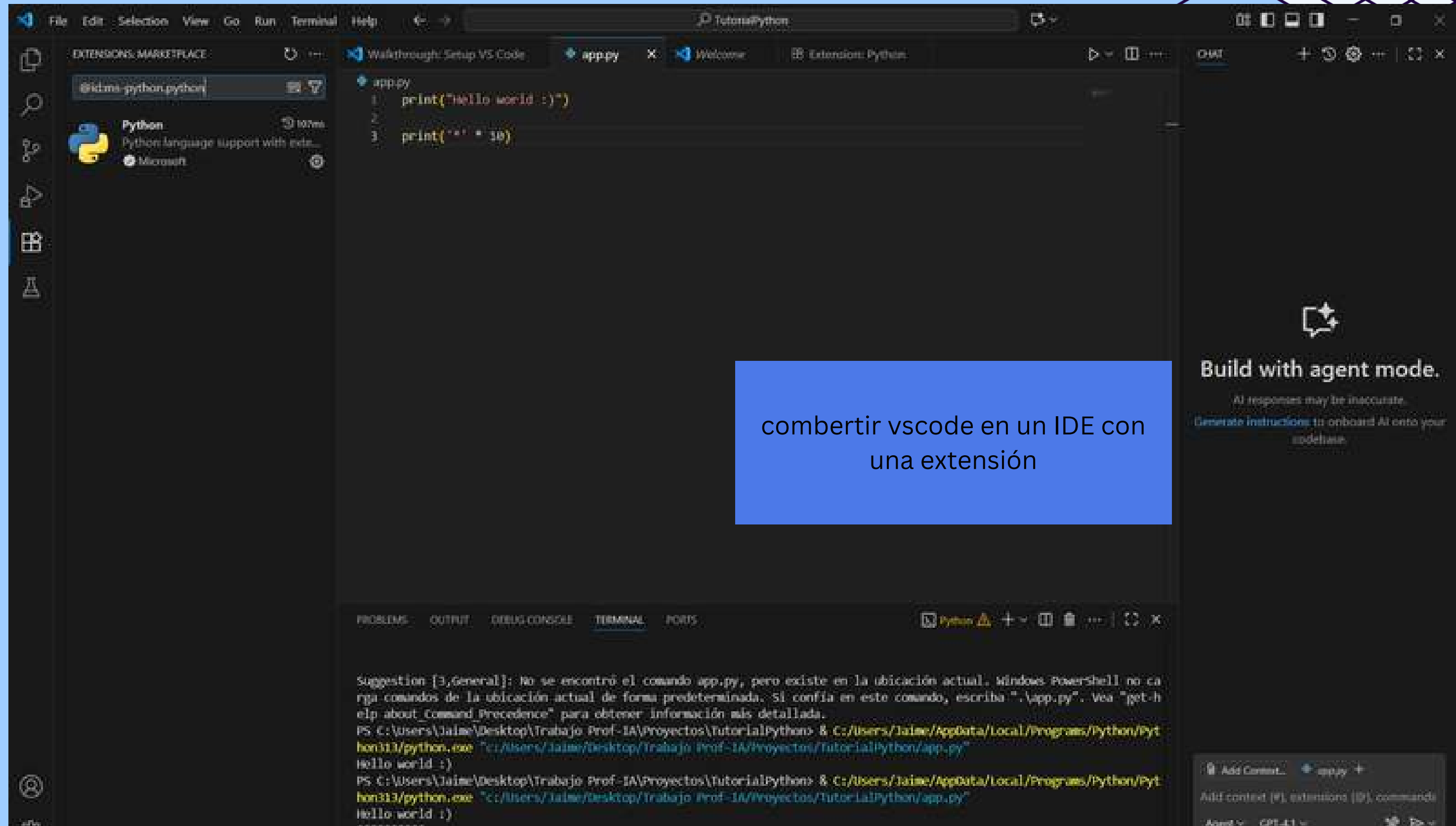
PASO 5

editor de video o IDE

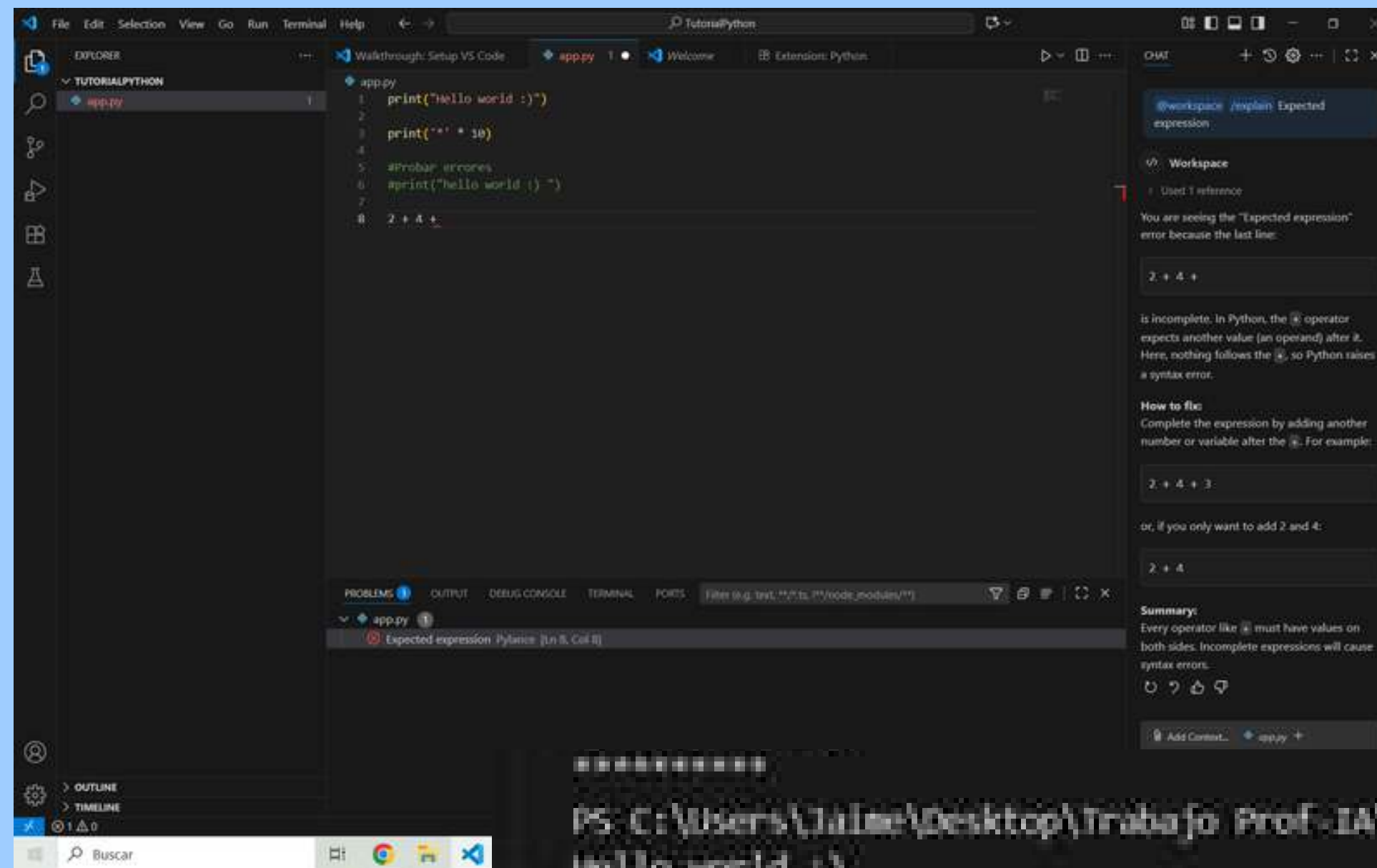
VScode - virtual studio
code.visualstudio.com



PASO 6



PASO 7



```
PS: C:\Users\Jaime\Desktop\Trabajo Prof-IA\Proyectos\tutorialPython> python app.py
Hello world :)
*****
```

PASO 8

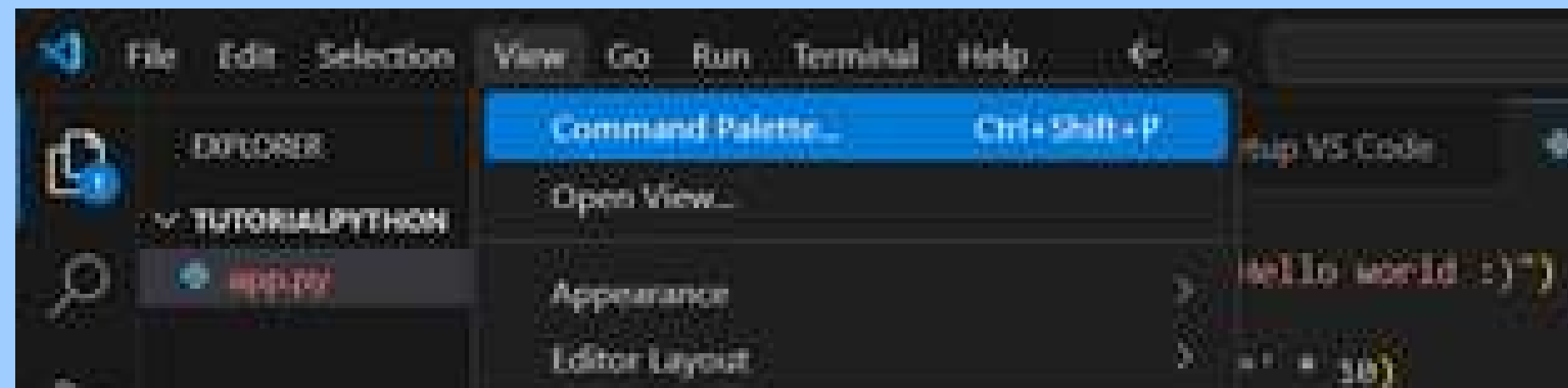
peps: documentacion de python

pep 8 : la principal

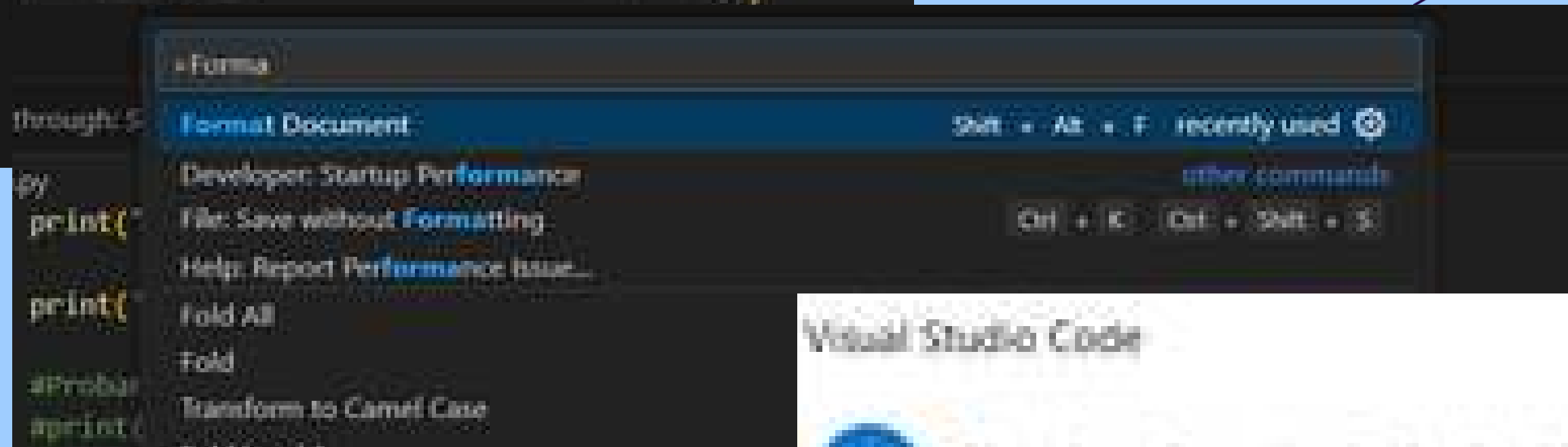
<https://peps.python.org/pep-0008/>



PASO 8.1

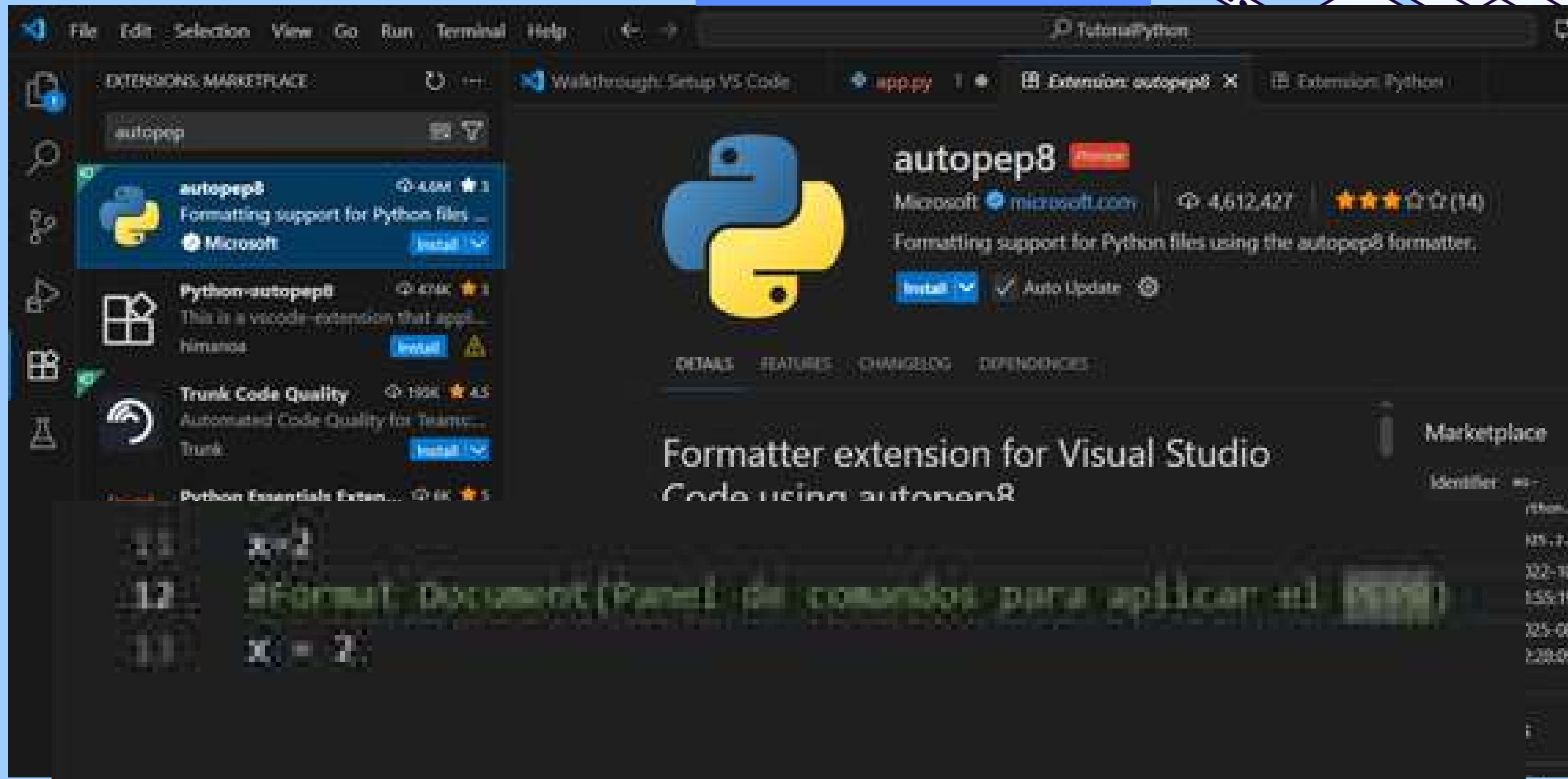


aplicar la pep8



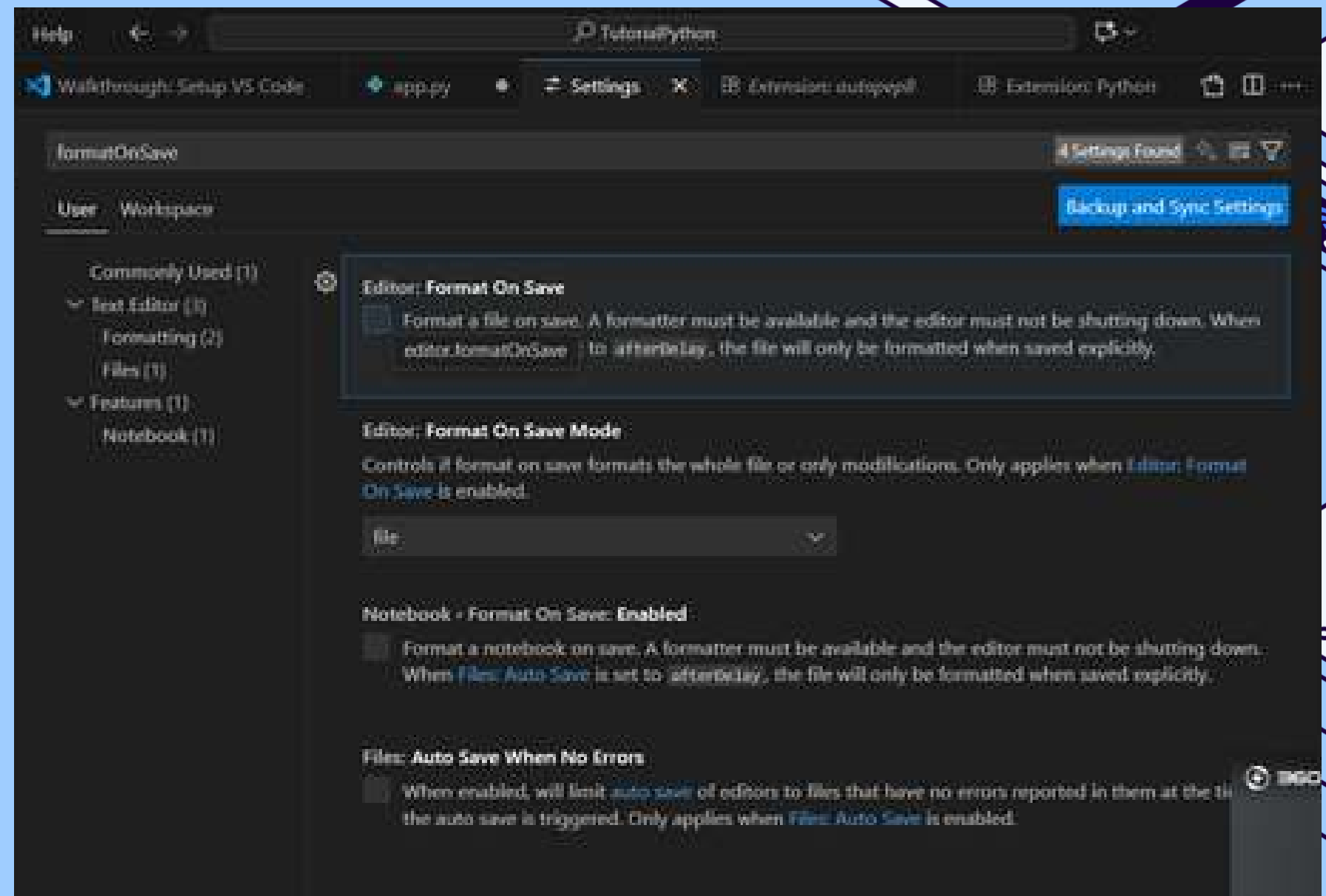
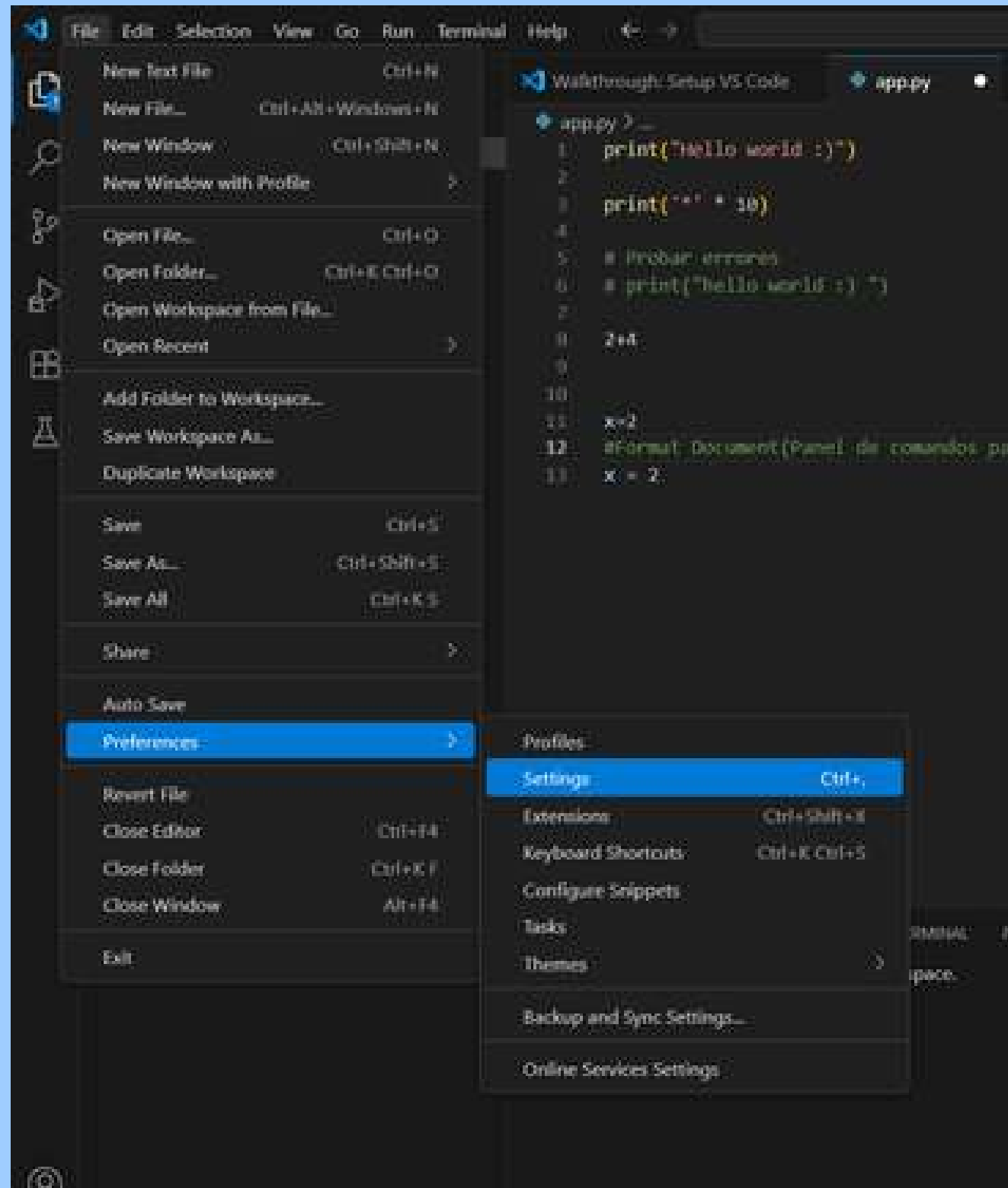
PASO 8.2

descargar la extension de pep8



PASO 8.3

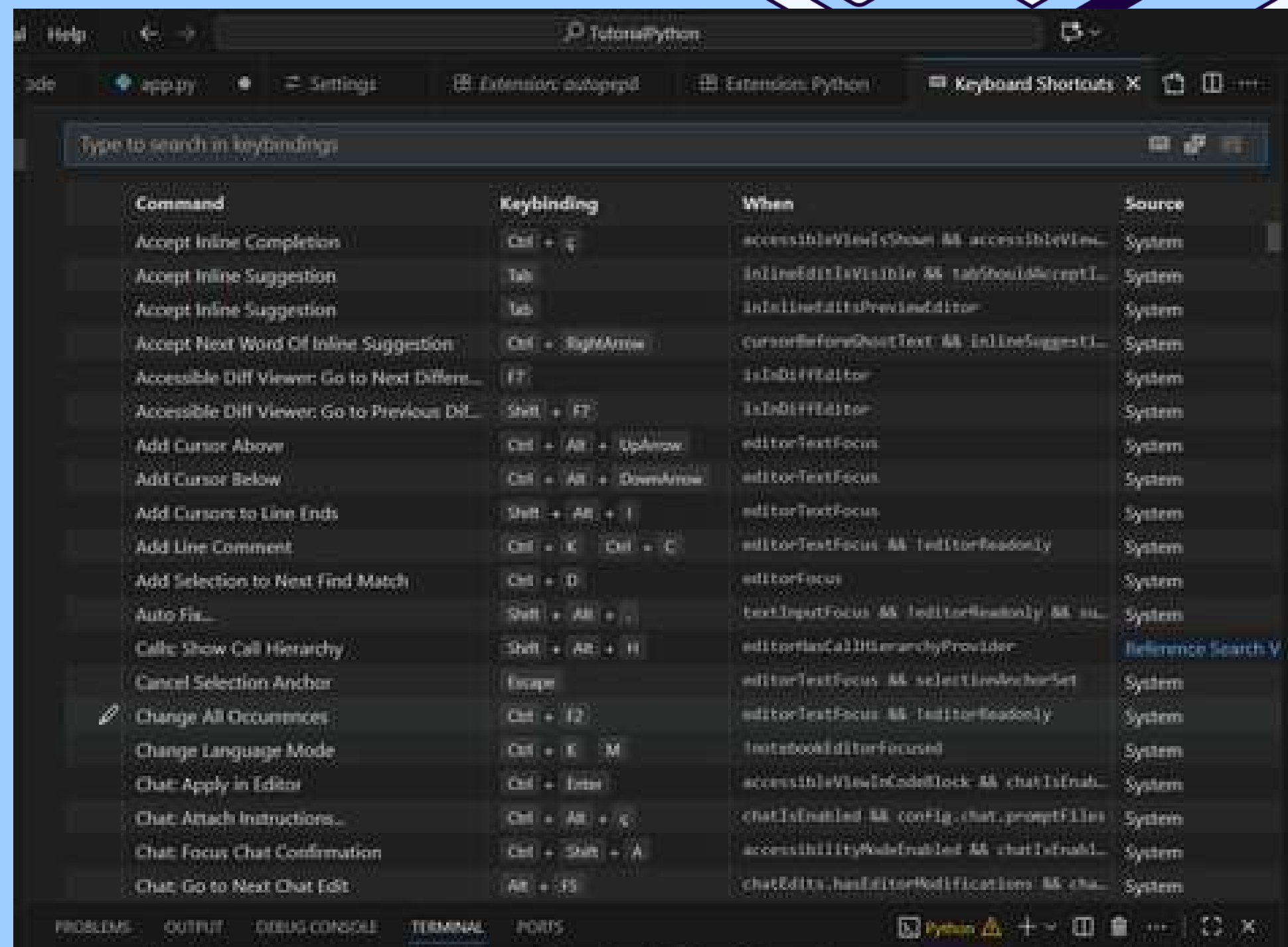
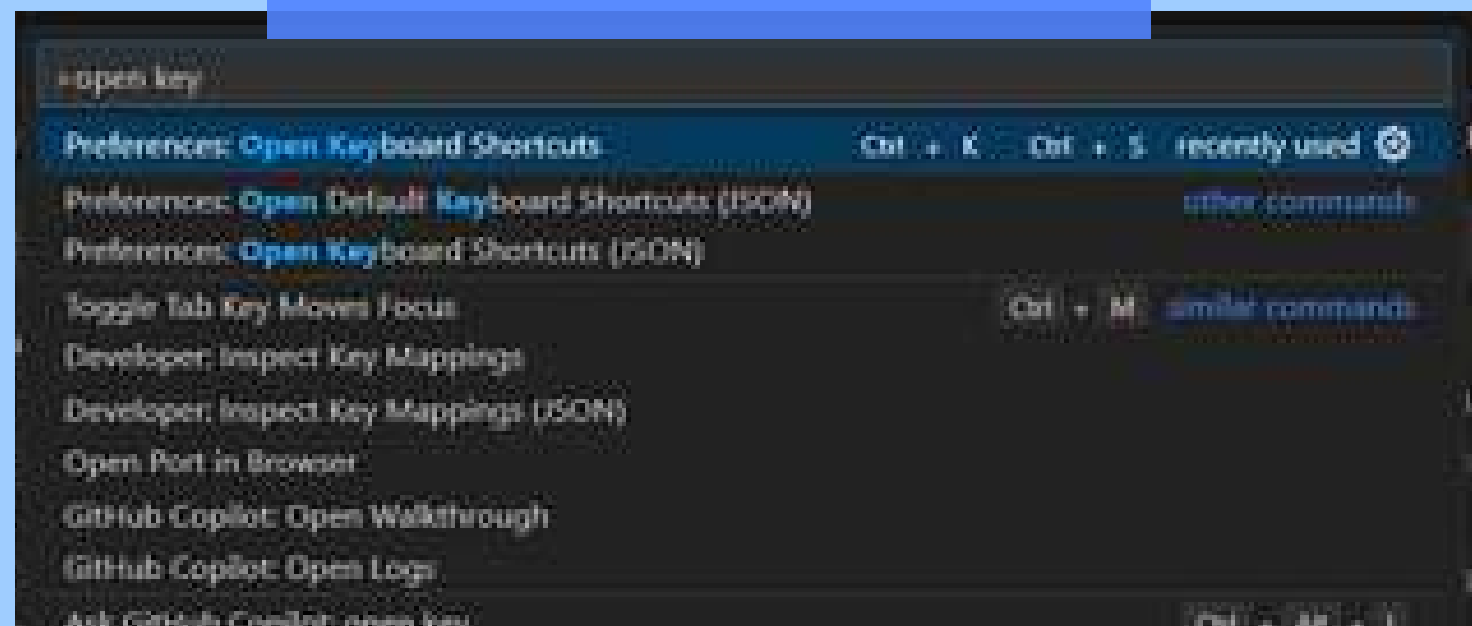
automatizar la pep8 para no tener
que llamarla



PASO 9

automatizar la pep8 para no tener que llamarla

shift+control+p



PASO 10

cpython jython ironpython
pypy



PASO 11

variable names(Codigo limpio): descriptivas y minimas y en minuscula,
sin_espacios_entre_palabras y los iguales con =
espacios



PASO 12

variables; bool, int, float, string



PASO 12.1

comillas

```
13
14 class = "godofredo"
15
16 correo_de_godofredo = ""
17 hola, soy Godofredo
18
19 No gusta programar en Python
20
21 Además, no gusta programar en JavaScript
22
23 Saludos|
24
25
26
```

PASO 12.2

comillas

`class = "good redo"`

```
print(len(class))  
  
print(class[0])  
print(class[-1])
```



The screenshot shows a Windows command prompt window with the following text:

```
PS C:\Users\Jaime\Desktop\Trabajo Prof-1A\Proyectos\TutorialPython> & C:/Users/Jaime/AppData/Local/Programs/Python/Python113/python.exe "c:/Users/Jaime/Desktop/Trabajo Prof-1A/Proyectos/TutorialPython/app.py"
Hello world :)
*****
9
8
0
PS C:\Users\Jaime\Desktop\Trabajo Prof-1A\Proyectos\TutorialPython>
```

The window title bar includes tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. The TERMINAL tab is active. The command prompt shows the execution of a Python script, which outputs "Hello world :)", followed by a separator line of asterisks and three lines of numbers (9, 8, 0).

PASO 12.3

comillas

```
26
27
28 print(len(clase))
29
30 print(clase[0])
31 print(clase[-1])
32 print(clase[0:3])
33 print(clase[3:])
34 print(clase[:3])
35 |
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python

```
8
9
PS C:\Users\Jaime\Desktop\Trabajo Prof-IA\Proyectos\TutorialPython> & C:/Users/Jaime/AppData/Local/Programs/Python/Pyt
hon313/python.exe "c:/Users/Jaime/Desktop/Trabajo Prof-IA/Proyectos/TutorialPython/app.py"
Hello world :)
*****
9
8
0
god
ofredo
god
PS C:\Users\Jaime\Desktop\Trabajo Prof-IA\Proyectos\TutorialPython>
```

Ln 35, Col 1 Spaces: 1

PASO 12.4

diagonal invertida

```
37 # \
38 # /
39 # \
40 # /
41 # \n
42 text = "\Hola, soy Gato\Fredo: \nMe gusta programar en Python\n"
43 print(text)
44
45 print("\n")
46
```

PROCESAL OUTPUT DEBUG-CONSOLE TERMINAL PORTS

```
9
8
0
god
ofredo
god

"Hola, soy Gato\Fredo:
Me gusta programar en Python\
```



PASO 12.5

```
17 first = "John"
18 last = "Doe"
19 full = f"{first} {last}"
20 print(full)
21 print(full.upper())
22 print(full.lower())
23 print(full.title())
24 print(full.strip())
25 print(full.find("a"))
26 print(full.replace("a", "A"))
27 print("John" in full)
28 print("Doe" in full)
29
30 print("\n")
```



PASO 12.5

variables numéricas y sus métodos

```
81
82 x = 10
83 y = 3.2
84 z = 1+4j
85 print(type(x), type(y), type(z))
86 print(isinstance(x, int))
87 print(isinstance(y, float))
88 print(isinstance(z, complex))
89 print(isinstance(z, float))
90 print(x + y) # suma
91 print(x - y) # Resta
92 print(x * y) # Multiplicación
93 print(x / y) # División
94 print(x // y) # División entera
95 print(x ** y) # Potencia elevada
96 print(x % y) # Módulo
97 print(abs(-x)) # valor absoluto
98 print(round(y)) # Redondeo
99 print(pow(x, y)) # Potencia
100 print(divmod(x, y)) # Cociente y resto
101
102 x += 3 # x = x + 3
103
```

PROBLEMA OUTPUT DEBUG-CONSOLE TERMINAL PORTS

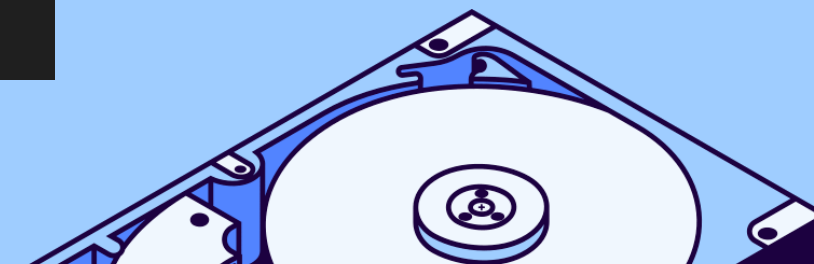
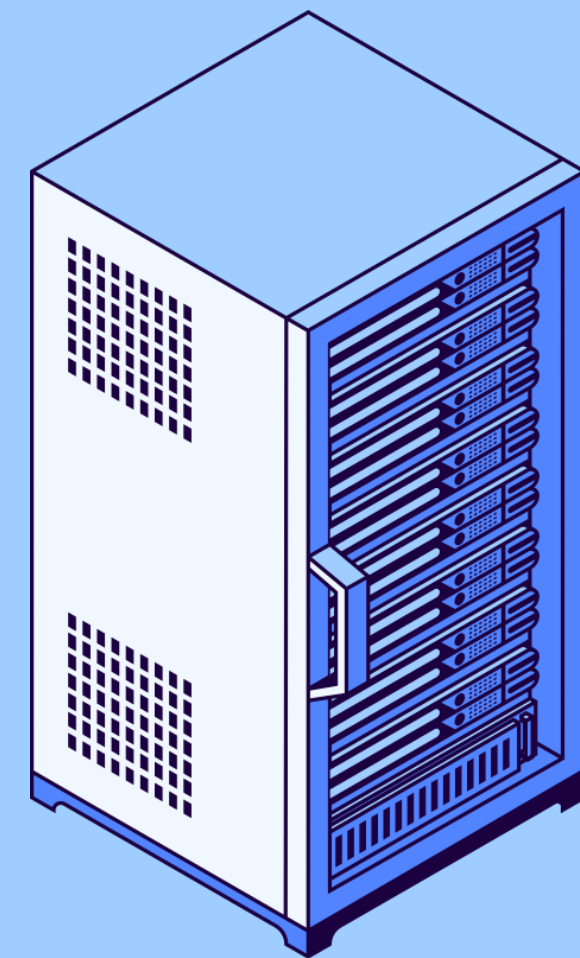
False
10
3.2
6.8
32.0
3.125
3.0
1584.893192461114
0.3099999999999997
10
3
1584.893192461114
(3.0, 8.309999999999999)



PASO 12.6

condicionales

```
16 temperature = 35
17 if temperature > 30:
18     print("Hace calor")
19 elif temperature > 20:
20     print("Hace buen tiempo")
21 else:
22     print("Hace frío")
23
24 age = 22
25 if age >= 18 and age < 65:
26     print("Eres un adulto")
27 if age < 18 or age >= 65:
28     print("No eres un adulto")
29
```



PASO 12.7.1

bucles simples

```
1 for number in range(1, 10):
2     print(number)
3
4 for number in range(1, 10):
5     print(number)
6
7 for number in range(1, 10):
8     print(number)
```



PASO 12.7.2

bucles anidados



PASO 12.7.3

bucles while

```
119 number = 100
120 while number > 0:
121     print(number)
122     number -= 1
123
```

```
132 command = ""
133 while command.lower() != "salir":
134     command = input("Escribe 'salir' para salir: ")
135 print("Has salido")
136
```

PROBLEMAS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Escribe 'salir' para salir: cacawete
Escribe 'salir' para salir: salir
Has salido
```

```
PS C:\Users\jaine\Desktop\Trabajo Prof-IA\Proyectos\TutorialPython>
```

PASO 12.7.3

bucles infinitos



```
130 command = ""
131 while True:
132     command = input("Escribe 'caracol' para salir: ")
133     if command.lower() == "caracol":
134         break
135     print("Has salido 🐌")
```

PROBLEMAS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Escribe 'caracol' para salir: cata
Escribe 'caracol' para salir: caasdad
Escribe 'caracol' para salir: CARACOL
Has salido 🐌
PS C:\Users\jaim\Desktop\Trabajo Prof-1A\Proyectos\TutorialPython>



PASO 12.8.1

funciones

```
135
136
137 def greet(): # Definición de la función
138     print("Hola")
139     print("¿cómo estás?")
140
141
142 greet() # Llamada a la función
```



PASO 12.8.2

```
144
145 def greet_with_name(name): # Definición de la función con parámetro
146     print(f"Hola {name}") # Uso del parámetro en la función
147     print("¿Cómo estás?")
148
149
150 greet_with_name("Pablo")
151
152
153 # Definición de la función con parámetro
154 def greet_with_optional_last_name(first_name, last_name=None):
155     if last_name:
156         # Uso del parámetro en la función
157         print(f"Hola {first_name} {last_name}")
158     else:
159         print(f"Hola {first_name}") # Uso del parámetro en la función
160     print("¿Cómo estás?")
161
162
163 # Llamada a la función con argumento opcional
164 greet_with_optional_last_name("Jaime", "Ciriacó")
165 greet_with_optional_last_name("Pablo")
```

PROBLEMS OUTPUT DEBUG-CONSOLE TERMINAL PORTS

Python

```
¿Cómo estás?
Hola Pablo
¿Cómo estás?
Hola Jaime Ciriacó
¿Cómo estás?
Hola Pablo
¿Cómo estás?
```

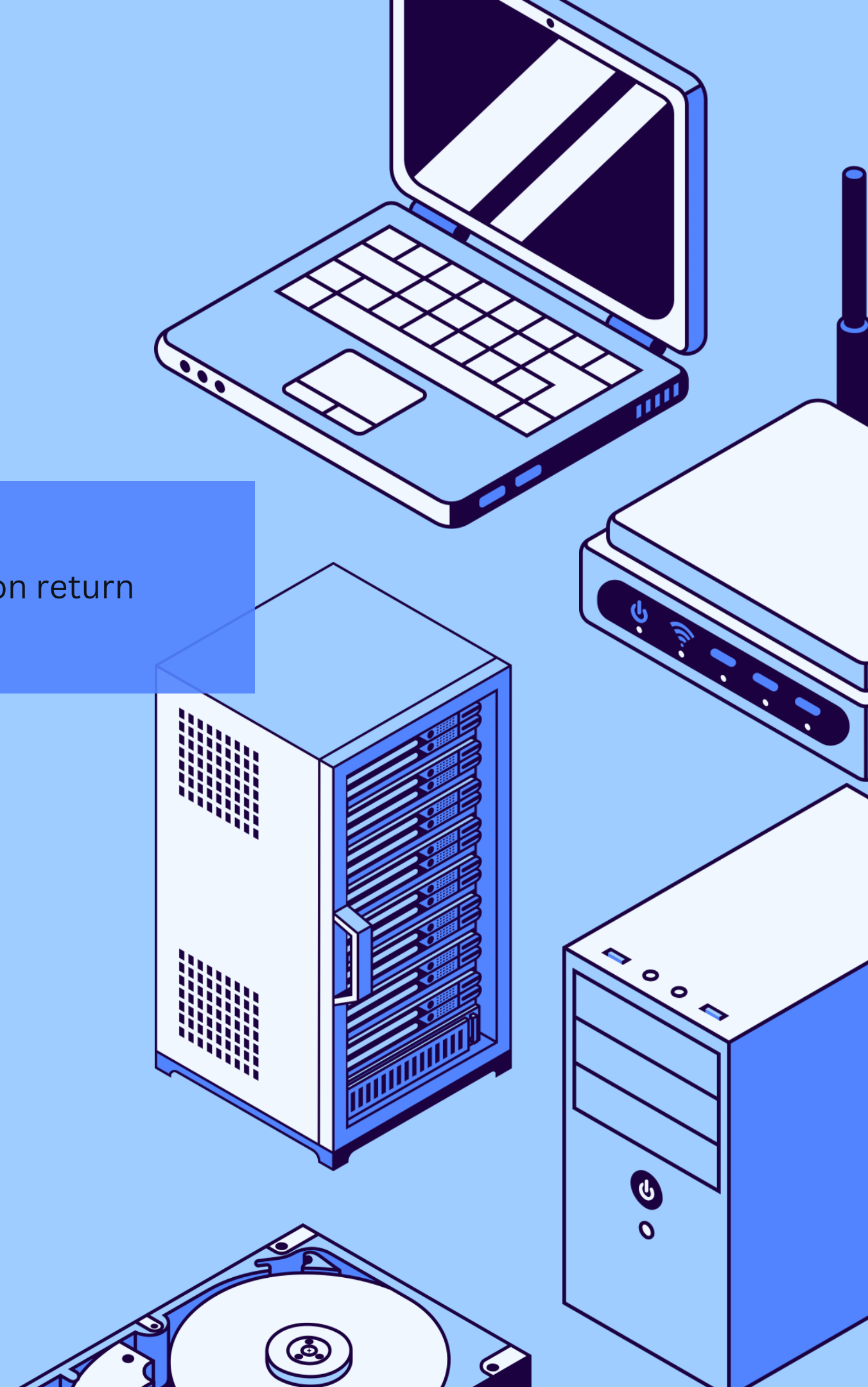
funciones con
argumentos(parametros)



PASO 12.8.3



funciones con return



PASO 12.8.3

```
175
176 def multiply(*numbers):
177     total = 1
178     print("multiplying: ")
179     for number in numbers:
180         total *= number
181         print(number, " ")
182     return total
183
184 result = multiply(2, 3, 4)
185
186 print("-", result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
multiplying:
2
3
4
- 24
```

argumentos con asterisco
(múltiples parámetros-colección
de parámetros)



Aunque hayamos dado nuestros primeros pasos de código con VS code, vamos a pasarnos a Anaconda, un entorno ya preparado y con una estructura más visual. Más adaptado para demostraciones y enseñanza.

Quien quiera podrá seguir desde aquí con VS code, incluso probar con Google Collab, la opción que nos da google trabajando desde la nube en sus TPU (Tensor Processing Units)





Anaconda Quick Start Environments

Jump into pre-configured environments by project or industry. Clean dependencies, faster development.

[Launch Your Environment](#)[Documentation](#)[Anaconda Blog](#)

All applications

on

base (root)

Channels

environment with hundreds of packages and store project files with persistent cloud storage.

[Launch](#)

greater).

[Launch](#)

greater).

[Launch](#)**JupyterLab**

4.3.4

An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.

[Launch](#)**Notebook**

7.3.2

Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.

[Launch](#)**Qt Console**

5.6.1

PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.

[Launch](#)